

1. El TAD MultiMap es una colección de pares (clave, valor), donde una clave puede estar asociada a muchos valores. Los siguientes son ejemplos de MultiMaps: $\{(1, a), (2, b), (1, c), (3, a)\}$, $\{(1, 3), (1, 4), (2, 6), (2, 7)\}$.

i) Completar la siguiente especificación del TAD MultiMap con la especificación de las operaciones *deleteAll* y *asMap*.

a) Nombre de tipo: MultiMap

Usa: Bool, Set, List

b) Operaciones:

tad MultiMap ($K : \text{Ordered Set}, V : \text{Set}$)

empty : *MultiMap* K V

put : *MultiMap* K $V \rightarrow (K, V) \rightarrow \text{MultiMap } K$ V

size : *MultiMap* K $V \rightarrow \text{Int}$

values : *MultiMap* K $V \rightarrow k \rightarrow \text{List } V$ -- lista los valores asociados a una clave

minKey : *MultiMap* K $V \rightarrow \text{List } V$ -- lista los valores asociados a la menor clave

delete : *MultiMap* K $V \rightarrow (K, V) \rightarrow \text{MultiMap } K$ V -- elimina un par (k,v)

deleteAll : *MultiMap* K $V \rightarrow K \rightarrow \text{MultiMap } K$ V -- elimina todos los valores asociados a una clave

asMap : *MultiMap* K $V \rightarrow \text{Set } K (\text{List } V)$ -- devuelve un conjunto con las claves
-- y sus valores asociados

c) Especificación algebraica.

$\text{size empty} = 0$

$\text{size (put } m \text{ (a, b))} = 1 + \text{size } m$

$\text{values empty } k = \text{nil}$

$\text{values (put } m \text{ (a, b)) } k = \text{if } a \equiv k \text{ then cons } b \text{ (values } m \text{ } k) \text{ else (values } m \text{ } k)$

$\text{delete empty (a, b)} = \text{empty}$

$\text{delete (put } m \text{ (c, d)) (a, b)} = \text{if } a \equiv c \wedge d \equiv b \text{ then } m \text{ else put (delete } m \text{ (a, b)) (c, d)}$

d

1. Suponiendo una representación de secuencias con árboles binarios, definidos con el siguiente tipo de datos:

data *Tree* $a = E \mid L \ a \mid N \ \text{Int} \ (\text{Tree } a) \ (\text{Tree } a)$

donde se almacenan los tamaños de los árboles en los nodos y el recorrido inorder del árbol da el orden de los elementos de la secuencia. definir en Haskell, de manera eficiente, paralelizando cuando sea posible, las siguientes operaciones sobre secuencias:

a) La función *concat* :: *Tree* (*Tree* a) $\rightarrow \text{Tree } a$, que concatena una secuencia de secuencias. Por ejemplo,

$\text{concat } \langle \langle 5, 2, 3 \rangle, \langle 6 \rangle, \langle 8, 0 \rangle \rangle = \langle 5, 2, 3, 6, 8, 0 \rangle$

b) La función *subsequence* :: *Tree* $a \rightarrow \text{Int} \rightarrow \text{Int} \rightarrow \text{Tree } a$, que dada una secuencia s y dos enteros i y j , devuelve la subsecuencia de s que está entre los índices i y j . Por ejemplo,

$\text{subsequence } \langle 1, 2, 3, 4, 5, 6 \rangle \ 2 \ 4 = \langle 2, 3, 4 \rangle = \langle 3, 4, 5 \rangle$ (índices en la seq en 0)

2. Usando las operaciones del TAD Secuencia definir las siguientes funciones:

a) *uniqify* :: *Seq* $\text{Int} \rightarrow \text{Seq } \text{Int}$, que elimina los elementos duplicados de una secuencia de enteros. Esta función puede definirse con profundidad en $O((\lg n)^2)$, donde n es la longitud de la secuencia.

b) *aa* :: *Seq* $\text{Char} \rightarrow \text{Int}$, que dada una secuencia de caracteres cuenta la cantidad de subsecuencias contiguas "aa" que contiene. La subsecuencia "aaa" debe sumar 2. Por ejemplo,

$\text{aa } \langle a, c, d, e, a, a, f, a, a, a \rangle = 3$

La función puede definirse con profundidad en $O(\lg n)$, siendo n la longitud de la secuencia.

c) Calcular el trabajo y la profundidad de las funciones definidas en los apartados anteriores.